SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Code of Conduct:

I NISHA K (192411231) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
NISHA K

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Q3–Q4 — Status code correctness & the fix

Q3 — DOES 200 OK MATCH REALITY?

No. `200 OK` asserts the request succeeded and the body is valid, but a malformed `Content-Length` means the message framing itself is broken — the client can't reliably read the response body at all. The status code and the header block contradict each other, so the response as a whole cannot be trusted as "OK."

Q4 — CORRECTIONS

- Set `Content-Length` to the actual byte length of the response body
- If the body length isn't known upfront, drop `Content-Length` and use `Transfer-Encoding: chunked` instead
- Add server-side validation that rejects negative or non-numeric lengths before the response is sent
- If generation genuinely failed, return `500 Internal Server Error` instead of `200 OK`

FIELD	AS SENT	VERDICT
Status line	200 OK	Misleading given the framing error
Content-Length	-150	Invalid — must be ≥ 0
Content-Type	text/html	Valid
Connection	keep-alive	Valid

LIVE DEMO — COMPUTE A CORRECT CONTENT-LENGTH

Paste or type a response body

```
<html><body>Hello</body></html>
```

Validate & fix header

```
before: Content-Length: -150 (invalid)
after:  Content-Length: 31 (matches actual body size)
status: HTTP/1.1 200 OK — now consistent with a well-framed body
```

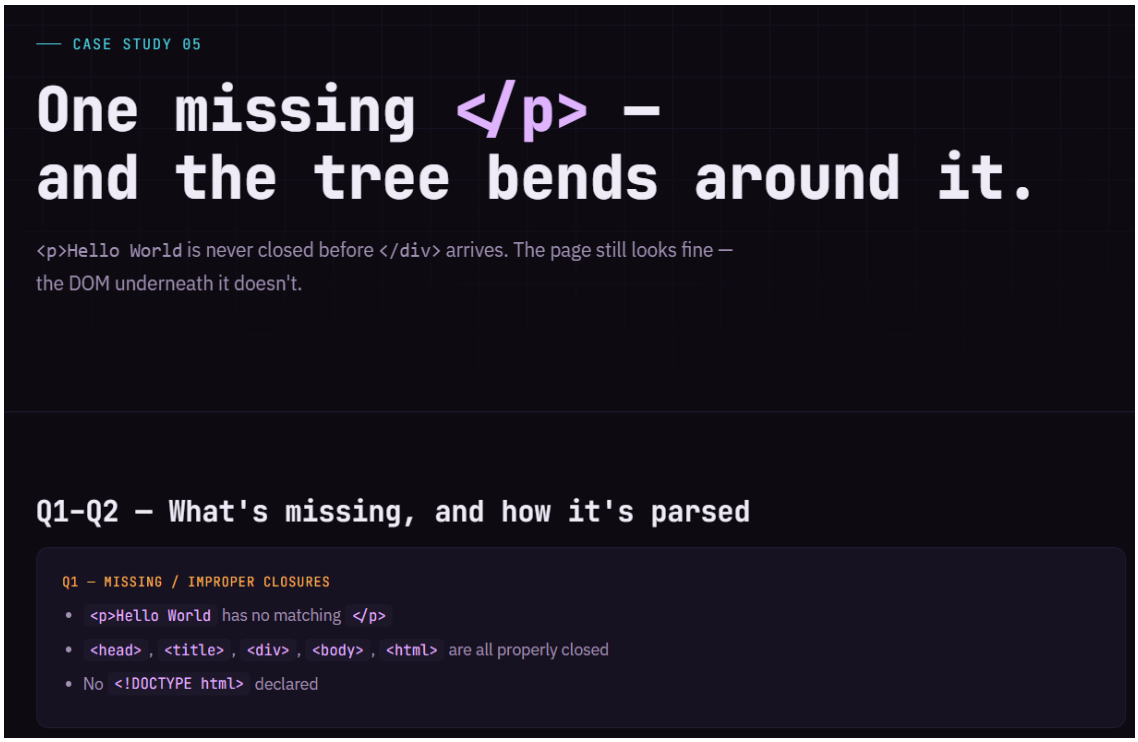
2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.



Q2 — How the browser interprets it

The HTML parser treats `<p>` as implicitly closed the moment it meets an element that can't legally live inside a paragraph. Here that's the closing `</div>` tag: the parser ends the open `<p>` right before it, then closes the `<div>` as written. The visible text is unaffected, but the parser inserted a closing tag that was never in the source.

Source — as written

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Corrected

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

Q3–Q4 — Rendering impact & the fix, visualized

Q3 — Impact on rendering & layout

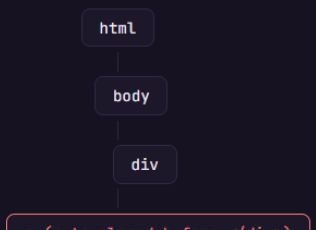
Visually, nothing looks broken — text still appears where expected. The real damage is structural: the parser-inserted close means `div > p` selectors, sibling combinators, and JS DOM traversal (`childNodes` , `closest()`) all see a shallower, differently-shaped tree than the source implied. Adding a sibling after the paragraph later would land in the wrong parent.

Q4 — Correction

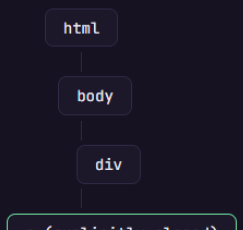
Close every element explicitly — `</p>` before `</div>` — and add a `<!DOCTYPE html>` and `<meta charset>` so the document parses in standards mode with an unambiguous, source-matching tree.

Live demo — the tree the parser actually builds

As written — parser-repaired tree



Corrected — matches the source exactly



3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

Q3–Q4 — Risk in GET, and secure practice

Q3 — SECURITY RISKS OF GET FOR AUTH

- Stored in plaintext in browser history and autocomplete
- Logged in plaintext by web servers, load balancers, and CDNs
- Leaked to third-party sites via the `Referer` header on outbound links
- Trivially shared or bookmarked with the password embedded
- Visible over-the-shoulder in the address bar itself

Q4 — BEST PRACTICES FOR SECURE LOGIN

- `POST` over **HTTPS** — POST alone doesn't encrypt anything, TLS does
- Hash + salt passwords server-side (bcrypt/argon2), never store or log them raw
- Session cookies marked `HttpOnly; Secure; SameSite=Strict`
- Rate-limit and lock out repeated failed attempts; add CSRF tokens
- Offer multi-factor authentication for sensitive accounts

METHOD	URL EXAMPLE	DATA VISIBILITY	SECURITY LEVEL
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST (+ HTTPS)	/login	Hidden in encrypted body	High

LIVE DEMO — EXPOSURE COMPARISON

GET POST

```
GET /login?user=admin&pass=1234 HTTP/1.1
→ logged in plaintext by server, proxy, and browser history
```

LIVE DEMO — EXPOSURE COMPARISON

GET POST

```
POST /login HTTP/1.1 (over TLS)
Content-Type: application/x-www-form-urlencoded

user=admin&pass=••••  → encrypted in transit, absent from the URL and history
```

SECURE LOGIN CHECKLIST

- ✓ POST over HTTPS — never GET, and never plain HTTP
- ✓ Passwords hashed + salted server-side, never logged raw
- ✓ Session cookies: HttpOnly, Secure, SameSite=Strict
- ✓ Rate limiting, account lockout, and CSRF protection on the login route

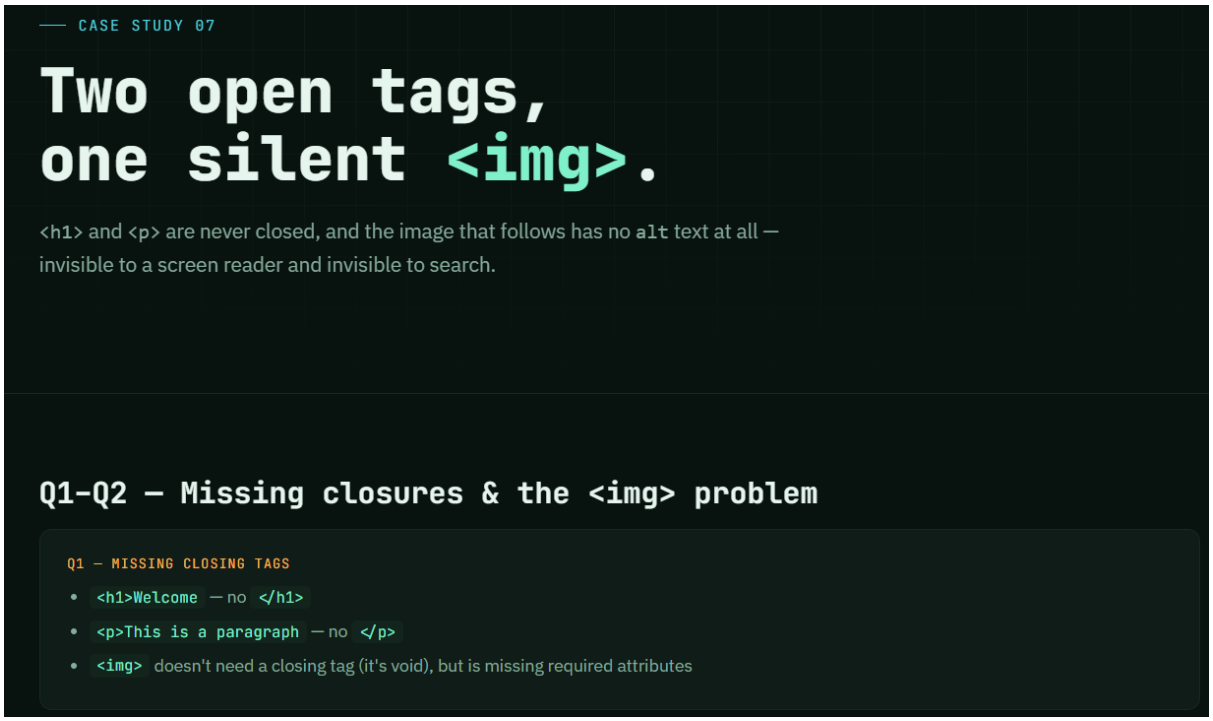
4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.



Q2 — ISSUES WITH THE <IMG> TAG

- No `alt` attribute — a screen reader has nothing meaningful to announce
- No `width` / `height` — the browser can't reserve space, causing layout shift as the image loads
- `src="image.jpg"` is a relative path with no context on whether it resolves correctly

BEFORE

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

CORRECTED

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>
<img src="image.jpg"
     alt="Description of the image"
     width="600" height="400">
</body>
</html>
```

Q3–Q4 — How browsers recover, and the full fix

Q3 — HOW BROWSERS HANDLE THESE ERRORS

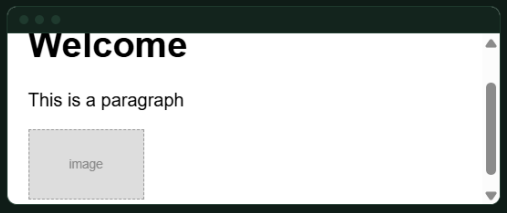
Because `<img>` is phrasing (inline) content, it doesn't force the open `<p>` to close — the parser lets it sit inside the paragraph. Both the open `<h1>` and `<p>` ultimately get implicitly closed when the parser reaches `</body>`. The rendered page looks intact, but a screen reader hits an unlabeled image with nothing to say.

Q4 — CORRECTED VERSION

Close `</h1>` and `</p>` explicitly, add a descriptive `alt` attribute to the image, and set `width` / `height` so the layout doesn't shift while the image loads.

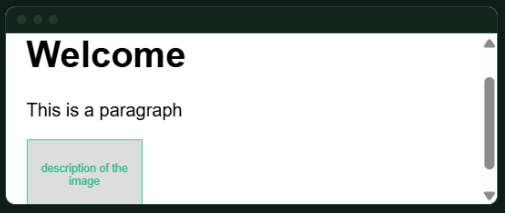
LIVE DEMO — WHAT ASSISTIVE TECH HEARS

BEFORE — NO ALT TEXT



`screen reader: "Welcome. This is a paragraph. image, unlabeled."`

AFTER — DESCRIPTIVE ALT TEXT



`screen reader: "Welcome. This is a paragraph. Image: description of the image."`

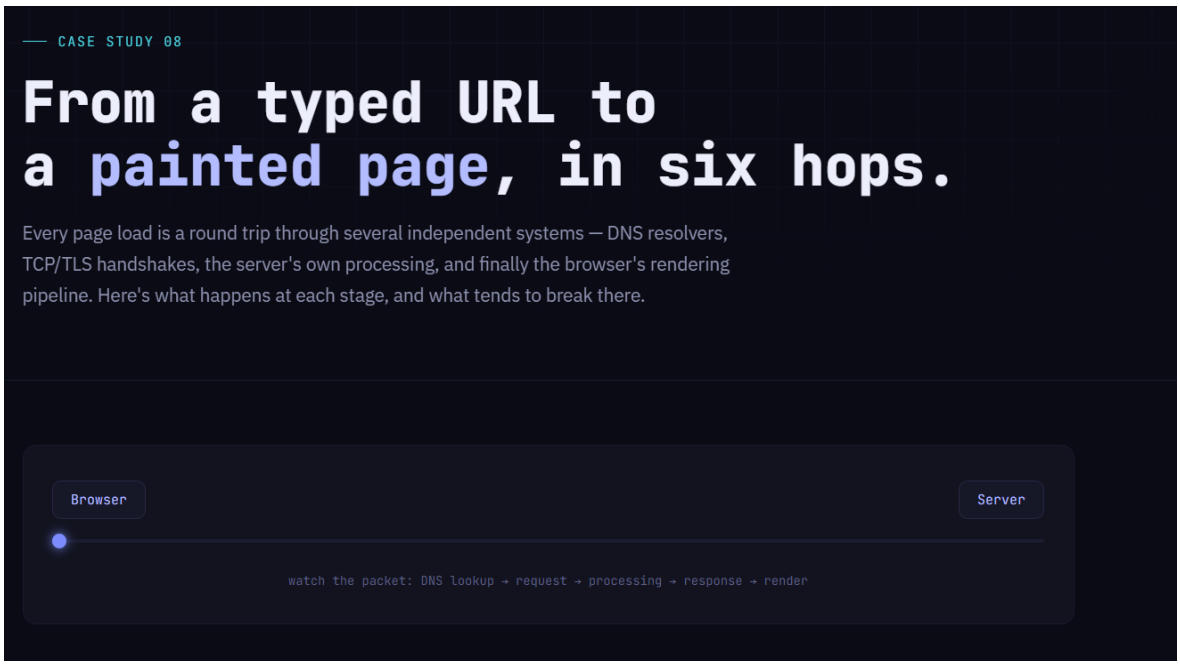
5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.



01 URL entered in the browser

URL parsing · HSTS check

The browser parses the scheme, host, path, and query, checks its HSTS list to decide whether to force HTTPS, and may serve the page instantly from cache if nothing has changed.

⚠ Typos in the domain, or a stale cache, are the most common failure here.

02 DNS lookup

DNS · UDP/53

The hostname is resolved to an IP address, checked first against the browser and OS cache, then a recursive resolver, then the authoritative nameserver chain if needed.

⚠ A slow or failing resolver is often the single biggest cause of a page that "won't even start" loading.

03 HTTP request is sent

TCP handshake · TLS handshake · HTTP/1.1 or 2

A TCP connection opens (SYN / SYN-ACK / ACK), TLS negotiates encryption if the scheme is HTTPS, and only then does the browser send the actual request line, headers, and any body.

⚠ Certificate errors and connection timeouts both happen before a single byte of HTTP is exchanged.

04 Server processes the request

Routing · App logic · DB / cache

The server routes the request to a handler, runs application logic, queries a database or cache as needed, and assembles a response with the correct status code and headers.

⚠ This is where a mismatched Content-Length or an incorrect status code — as in CS-04 — gets introduced.

05 Response is returned

Status line · Headers · Body

The server streams the status line, headers, and body back over the same connection (or a multiplexed HTTP/2 stream), and the browser begins reading it as bytes arrive.

⚠ A malformed header here can stall the connection even if the body itself is fine.

06 Browser renders the page

DOM · CSSOM · Layout → Paint

The browser parses HTML into a DOM and CSS into a CSSOM, combines them into a render tree, computes layout, and paints pixels — fetching additional resources like images and scripts along the way.

⚠ Malformed markup (CS-02, CS-05, CS-07) is repaired here silently, which is exactly what makes those bugs hard to spot.